USEI11 — Directed Line Upgrade Plan (Topological Sort)

Algorithm: RailwayUpgradeService.computeUpgradePlan() (Kahn's Algorithm)

Time Complexity: O(V + E)

Where:

- V = Number of vertices (Stations) in the graph
- E = Number of edges (TrackConnection) in the directed graph

Detailed Analysis:

1. **Build Internal Structures (**buildInternalStructures**)**
    a. Initialize inDegree and adjacencyList for all vertices: O(V)
    b. Iterate through all edges to populate adjacency list and increment inDegree: O(E)
    c. **Subtotal:** O(V + E)
2. **Kahn's Topological Sort (Main Loop)**
    a. Find initial vertices with inDegree = 0: O(V)
    b. **While** queue is not empty:
        i. Remove a vertex: O(1)
        ii. For each neighbor, decrement inDegree: O(1) per neighbor
        iii. Each edge is processed exactly once when its source vertex is dequeued.
    c. **Total edge processing:** O(E)
    d. **Total vertex processing:** O(V)
    e. **Subtotal:** O(V + E)
3. **Cycle Detection & Result Assembly**
    a. Compare size of topological order to V: O(1)

b. If cycles exist, find stations/edges involved: O(V + E) in worst case (visiting only cycle components).

c. **Subtotal:** O(V + E)

Overall Time Complexity:

O(V + E) + O(V + E) + O(V + E) = O(V + E)

The algorithm performs a **constant number of linear passes** over vertices and edges, resulting in **linear time complexity** relative to the graph size.

Space Complexity: O(V + E)

Breakdown:

- **Adjacency List:** Stores all vertices and their outgoing edges → O(V + E)
- inDegree **Map:** Stores an integer for each vertex → O(V)
- **Queue:** Can hold up to all vertices in worst case → O(V)
- **Topological Order List:** Stores all vertices in order → O(V)
- **Cycle result lists:** Up to O(V + E) in worst case.

Dominant Factor: Adjacency list storage → O(V + E).